

10A. AVL TREES

Balanced Binary Search Trees (AVL Trees)

Insertion

AVL trees (1)

See:

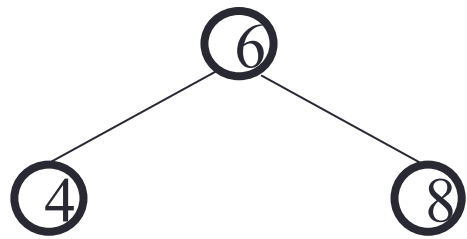
- G.M. Adelson-Velskii and E.M. Landis, "An algorithm for the organization of information." *Doklady Akademii Nauk SSSR*, 146:263–266, 1962 ([Russian](#)). [English](#) translation by Myron J. Ricci in *Soviet Math. Doklady*, 3:1259–1263, 1962.
- [Donald Knuth](#). *The Art of Computer Programming*, Volume 3: *Sorting and Searching*, Third Edition. Addison-Wesley, 1997. [ISBN 0-201-89685-0](#). Pages 458–475 of section 6.2.3: Balanced Trees. Note that Knuth calls AVL trees simply "balanced trees".

AVL Trees

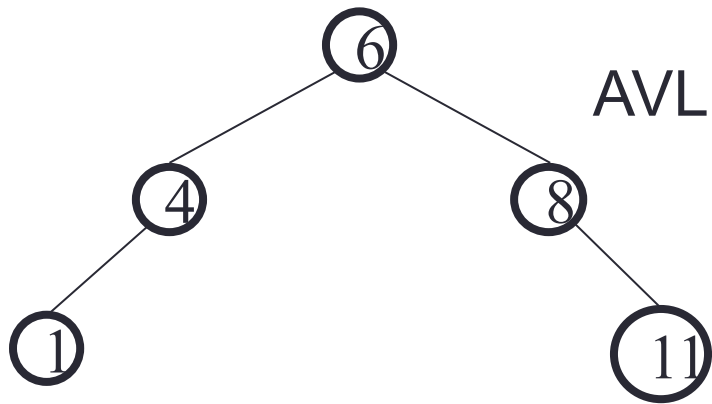
- Discovered in 1962, Adelson-Velskii and Landis
 - Trying to devise a better data structure for a chess engine
- Depth ensured to be $O(\log n)$
 - But not always perfectly balanced
- Recursive definition:
 - The sub-trees of every node differ in height by at most one.
 - Every sub-tree is an AVL tree.

Height of an empty tree is -1

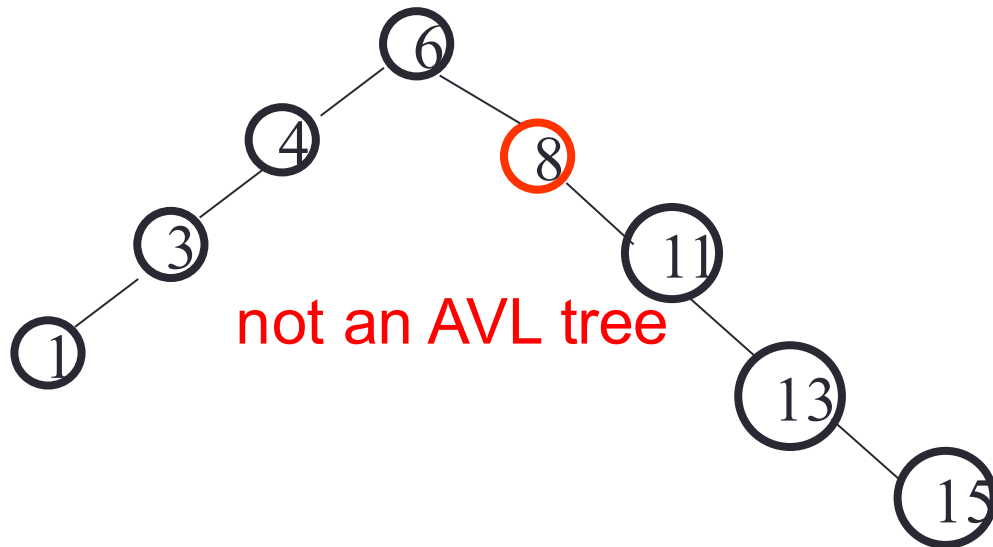
Height of a single node is 0



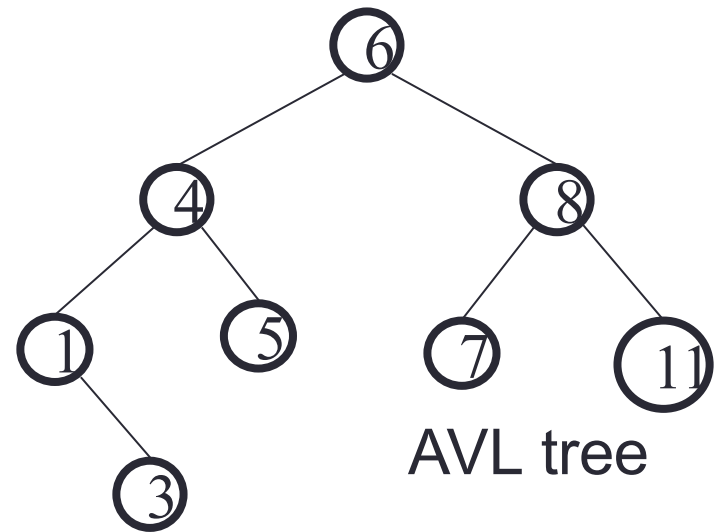
AVL tree



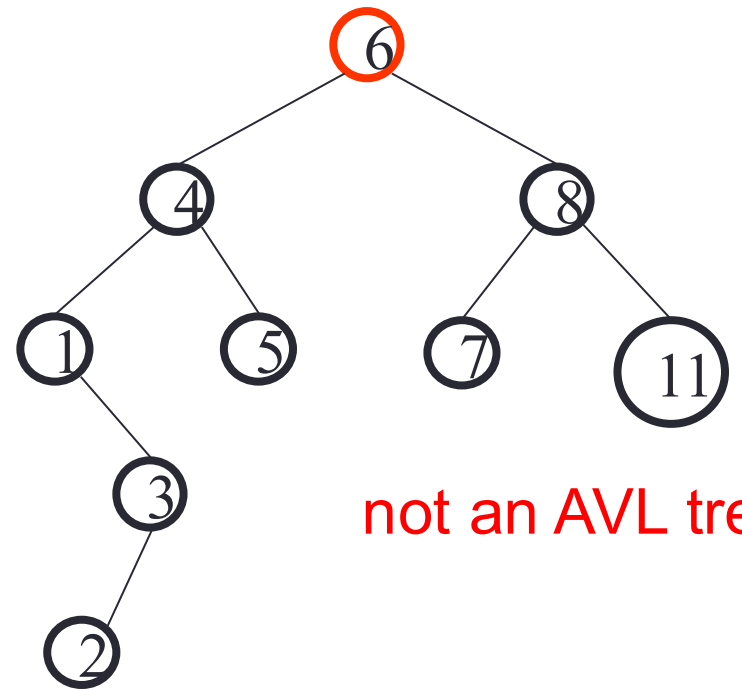
AVL tree



not an AVL tree



AVL tree



not an AVL tree

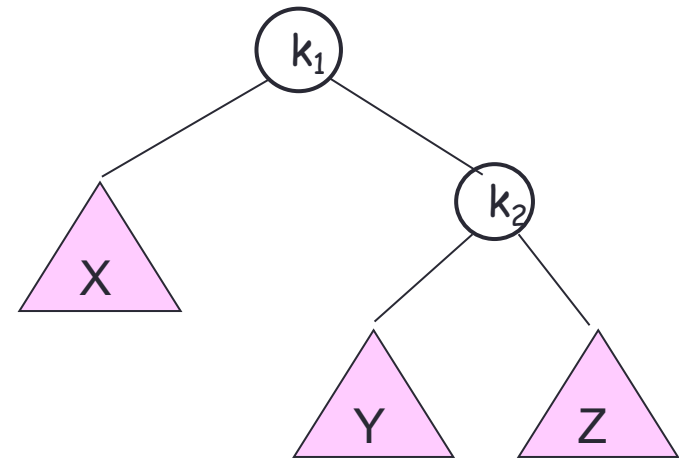
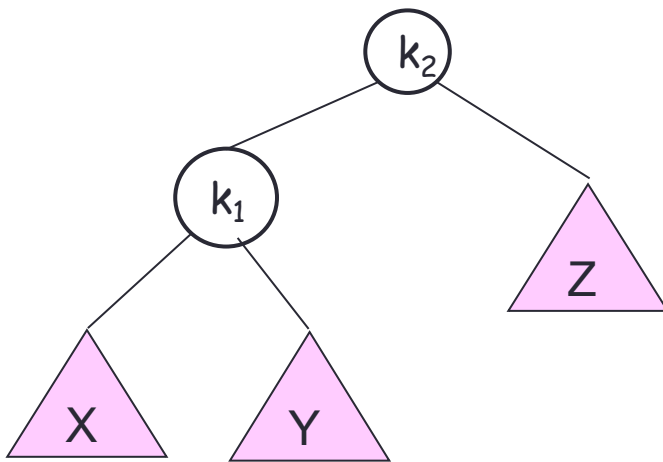
Operations on AVL trees

- Search, as for BSTs
- Insert is the same as for BST but
 - May have to fix the tree after insertion to restore the balance condition
 - Can always do this via a modification known as a rotation
 - Single rotation or double rotation
- Similarly for deletion

Note that the balance condition ensures that search, insert and delete operations are all $O(\log n)$, where n is the number of nodes

balancing is $O(1)$

The basic idea of single rotation



In each case:

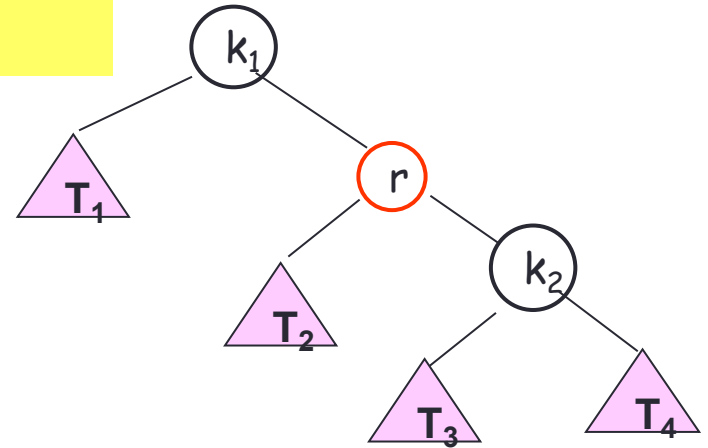
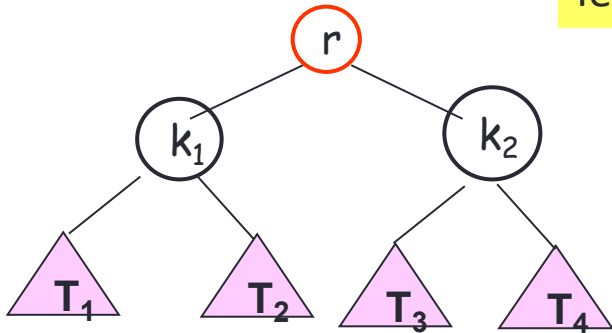
- $k_1 < k_2$,
- all elements in subtree X are smaller than k_1 ,
- all elements in subtree Z are larger than k_2 and
- all elements in subtree Y are in between k_1 and k_2

So:

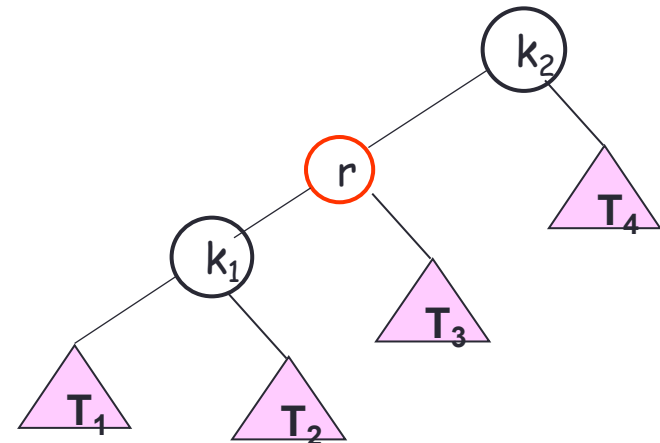
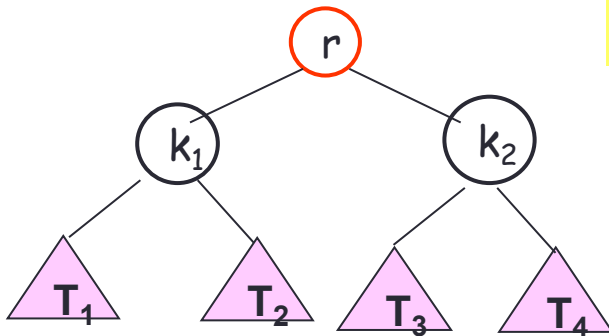
- both trees are BSTs for the same elements
- Conversion of one to the other is known as a *rotation*

To rebalance a binary search tree rooted at r

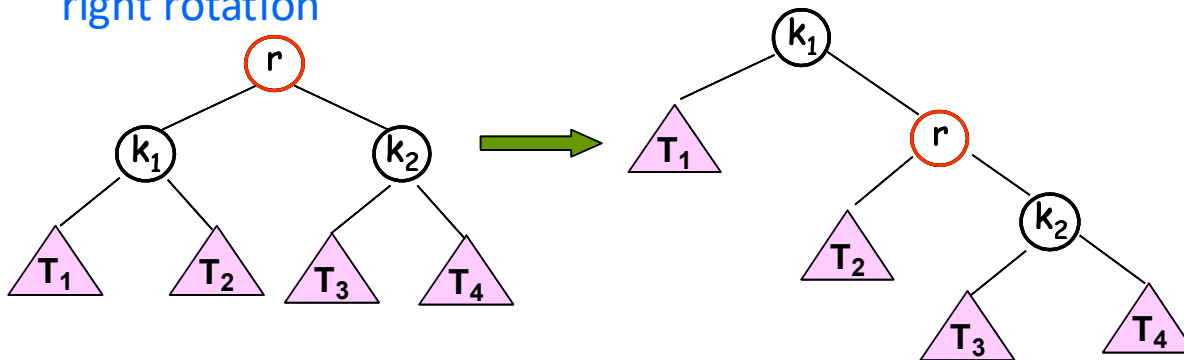
If too heavy on the left...right rotation



If too heavy on the right... left rotation



right rotation

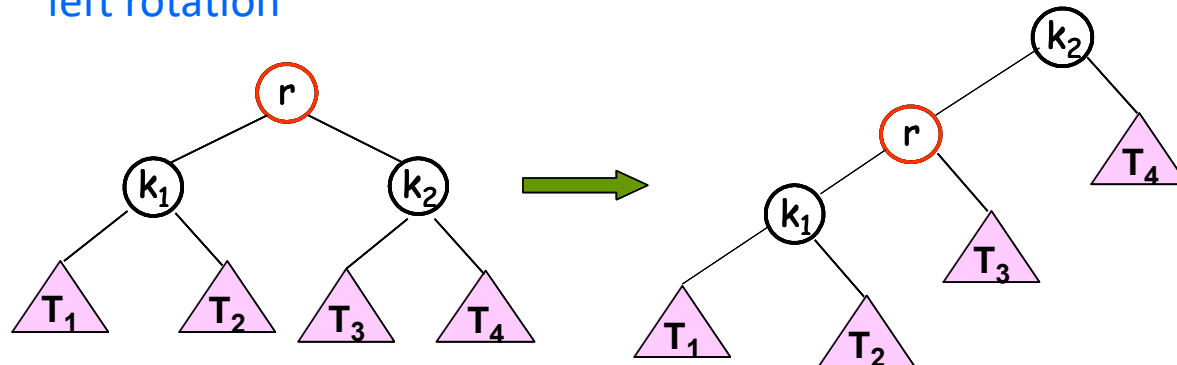


If node r is **too heavy on the left**, rotating about node r and left child:

Rotate the whole subtree rooted at r round one place to the right

Take the right subtree of new root and make it the left subtree of r

left rotation

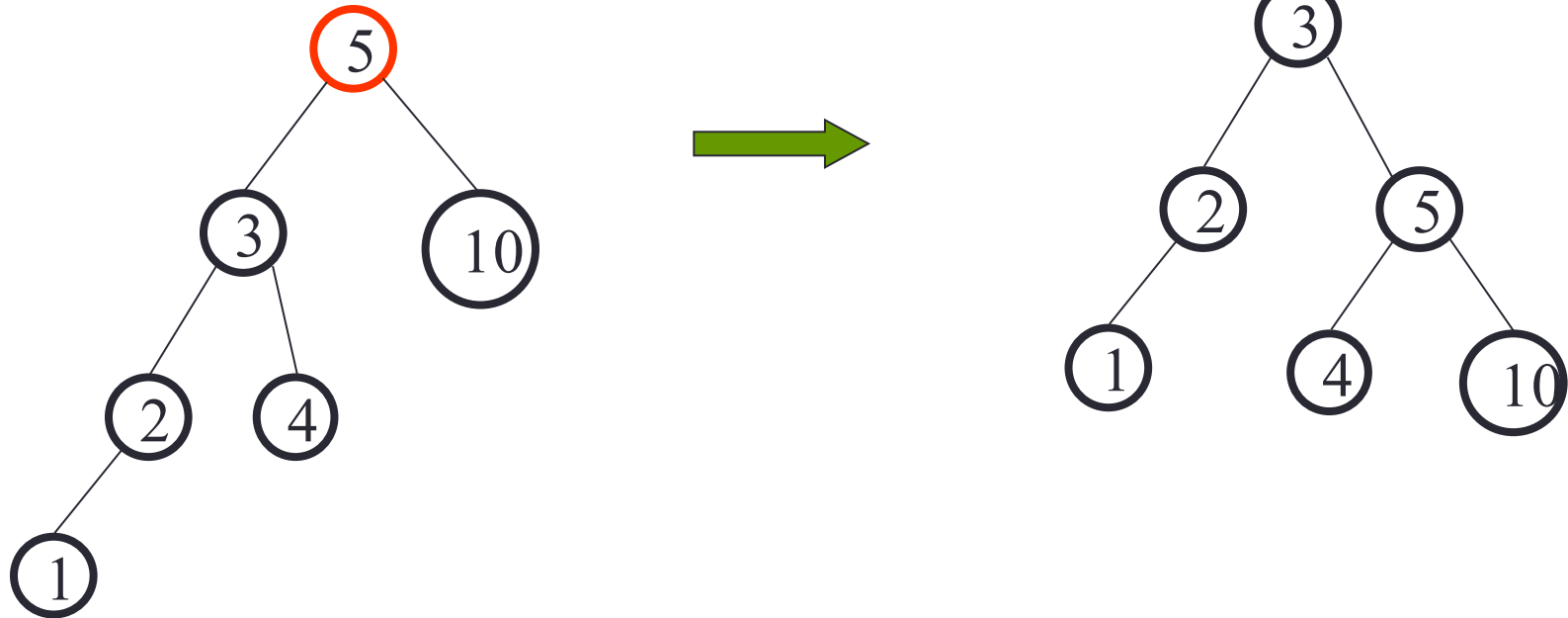


If node r is **too heavy on the right**, rotating about node r and right child:

Rotate the whole subtree rooted at r round one place to the left

Take the left subtree of new root and make it the right subtree of r

Example: right rotation

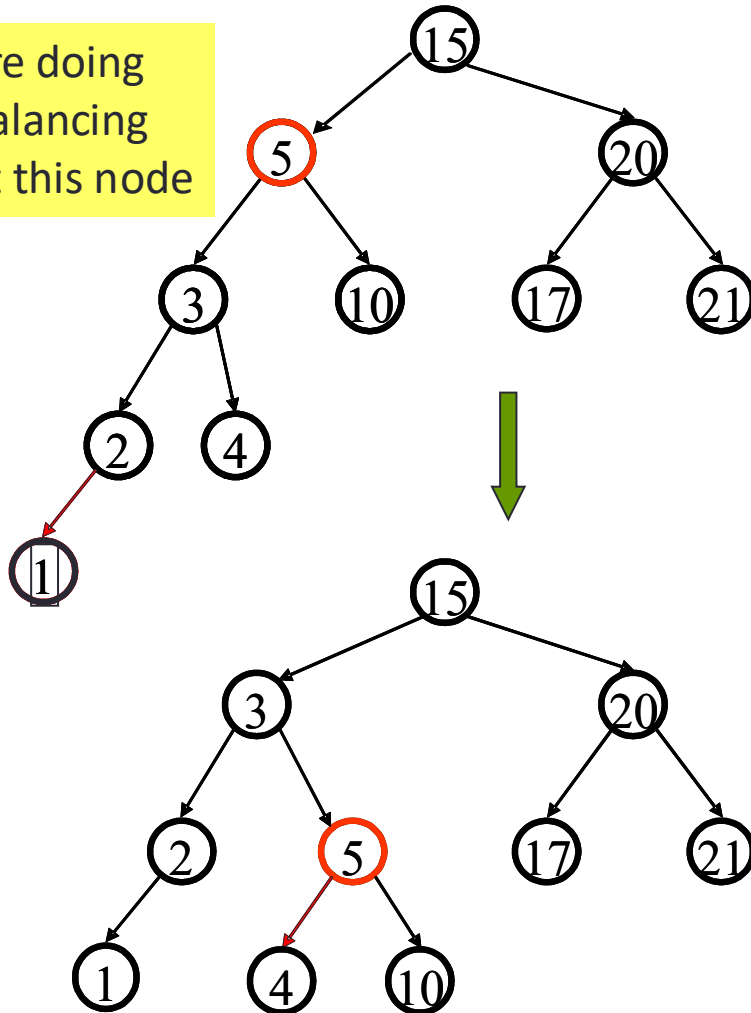


- Note that the first tree is not an AVL tree, but the second one is
- Rotation involves only a few pointer changes and allows us to fix the structure of a tree whilst preserving search tree property

Rotation continued

- Doesn't have to be done at the root of a tree; can be done at any node since that node is the root of some subtree.

We are doing
the balancing
about this node



Applying single rotation – a worked example

Start with an empty AVL tree and insert values 1 to 7 in sequential order then values 8 to 15 in reverse order

To maintain balance condition, every time you insert a new node:

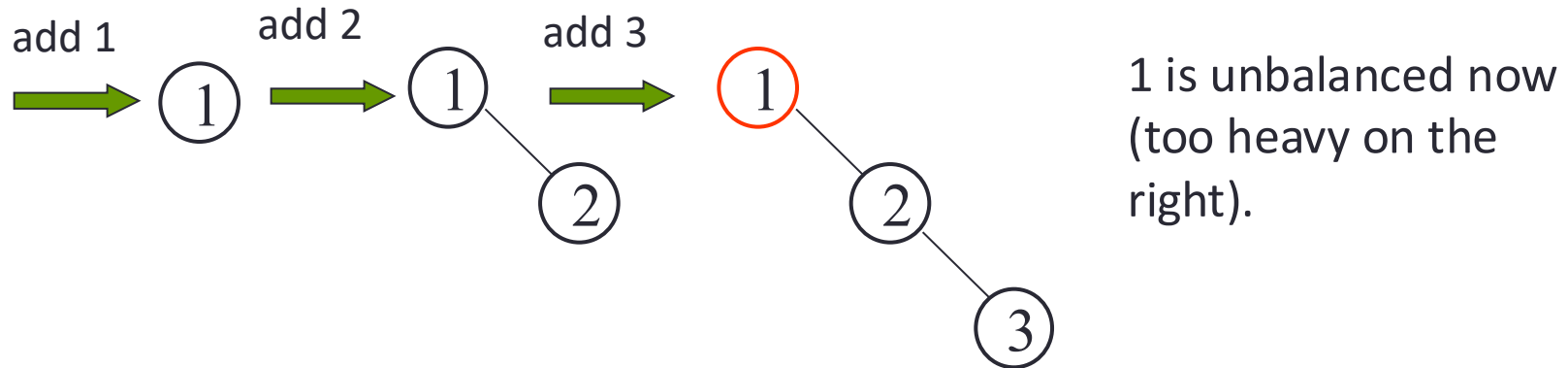
Starting at the node just inserted, travel up tree

- If we reach root node without finding a bad node, **we are done**
- If we reach bad node, do a rotation and **we are done**

Note we are balancing the **deepest unbalanced node**

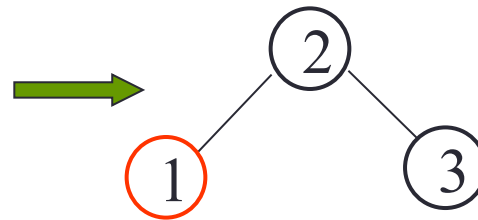
If tree is now **balanced**, continue.. (otherwise..what?)

Worked example (adding nodes containing 1..7 in order)

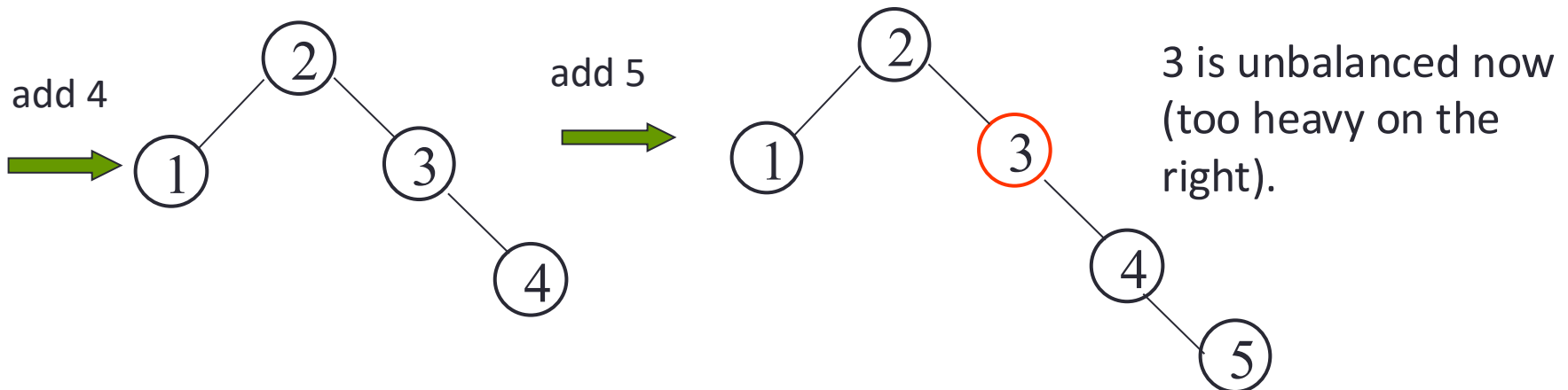


Left rotation.

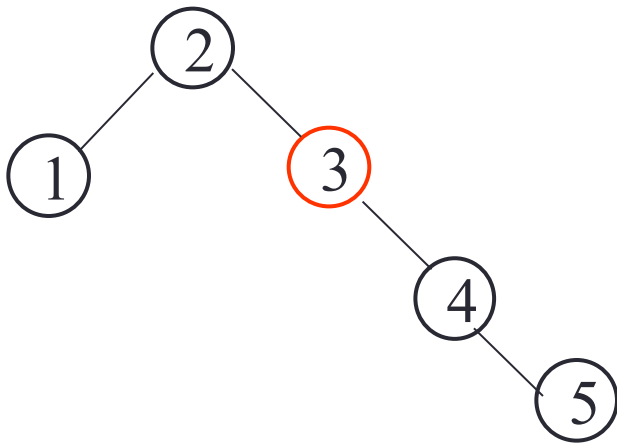
(Node 2 has no left subtree to move)



balanced? Yes
→ continue

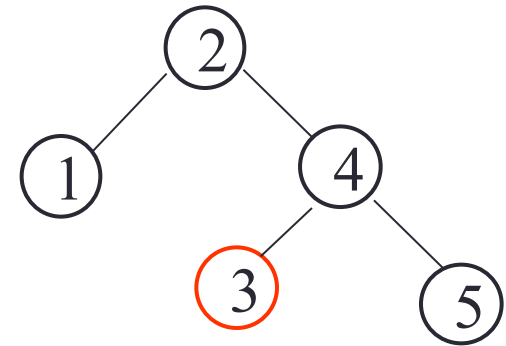


Worked example contd.



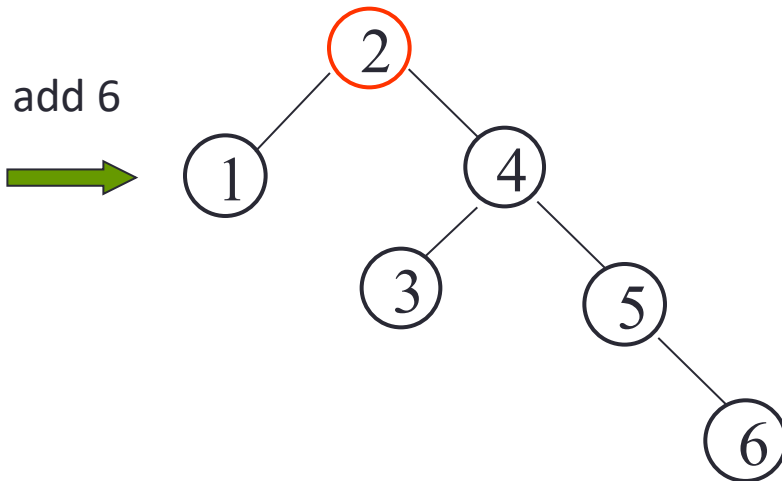
Left rotation.

(Node 4 has no left subtree to move)



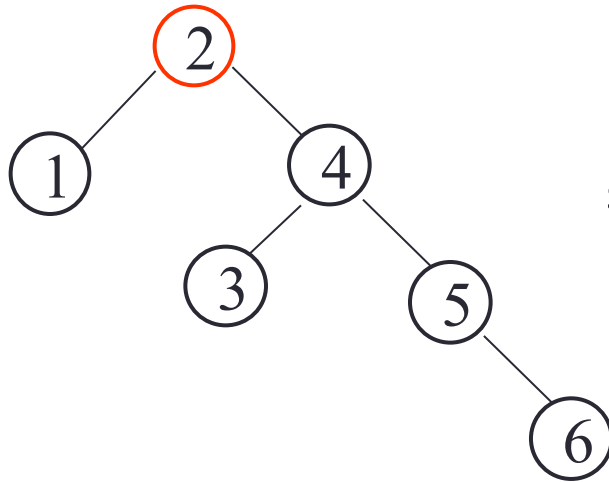
balanced? Yes

→ continue



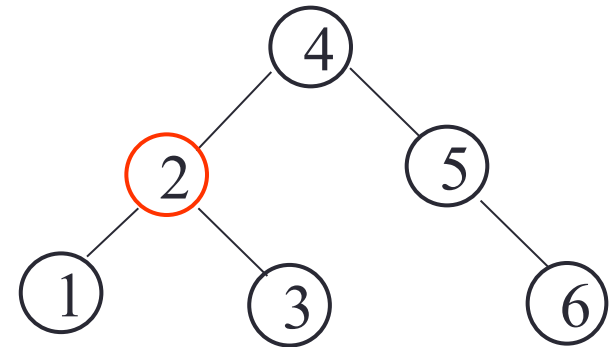
2 is unbalanced now
(too heavy on the right).

Worked example contd.



Left rotation.

(Node 4 **has** left subtree to move)

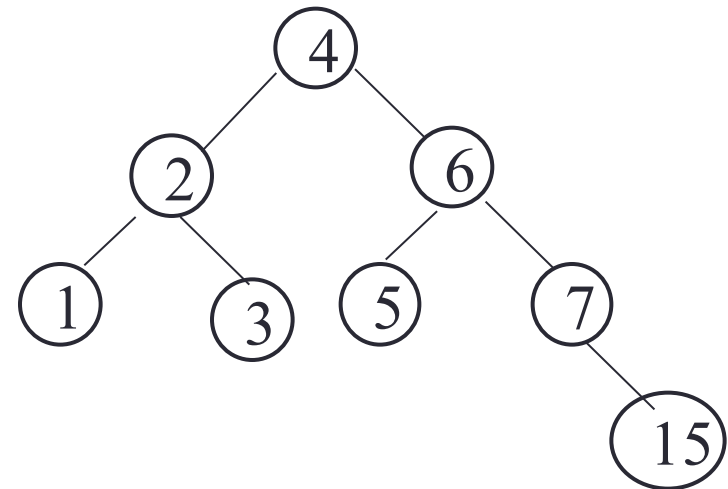


balanced? Yes

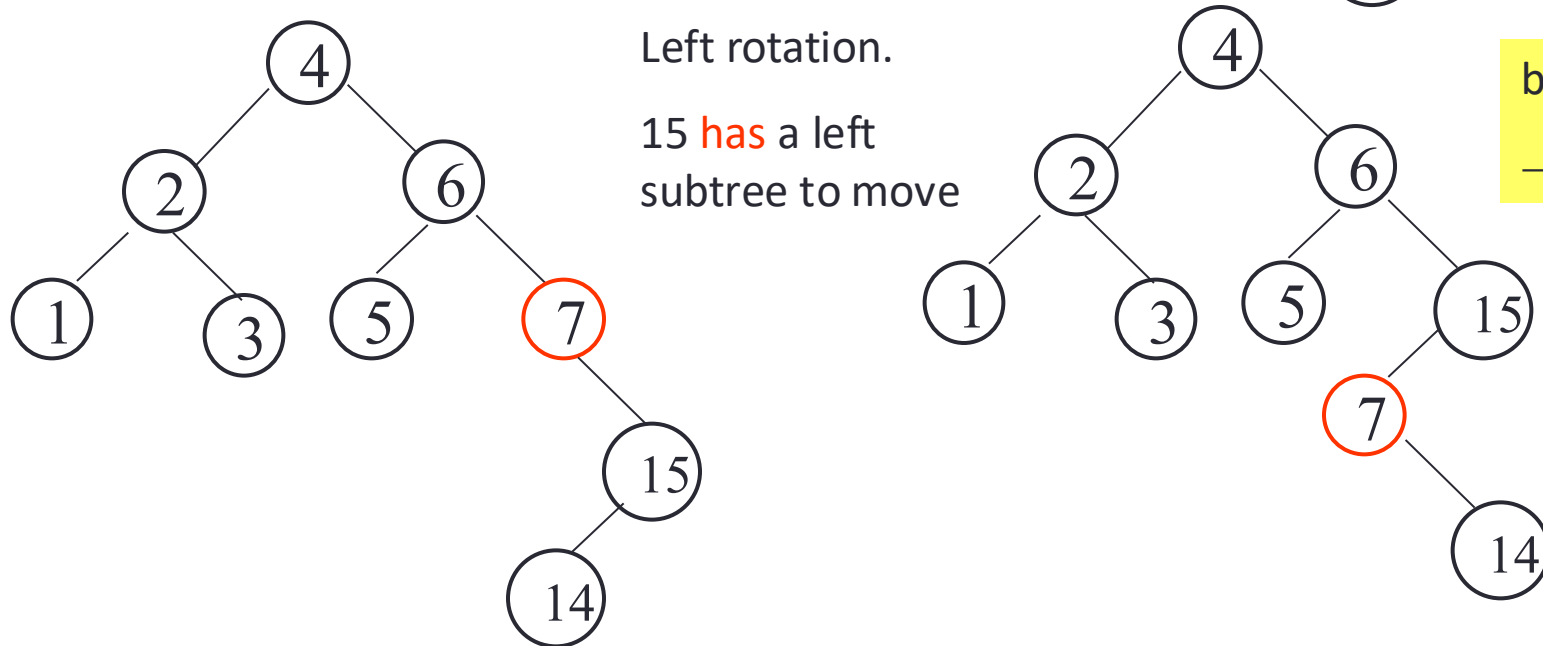
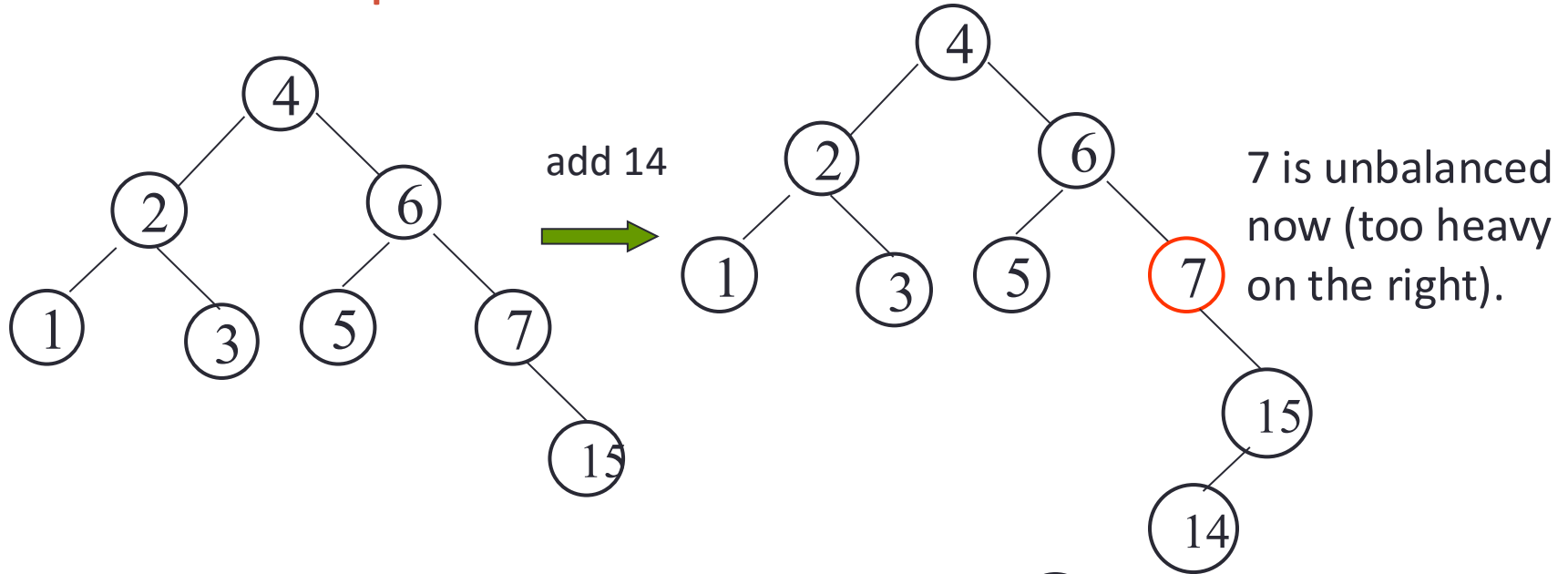
→ continue

Example for you: add the next two nodes, i.e. node containing the value 7 and node containing the value 15

add 7
and 15



Worked example contd.



balanced? No!
→ What now?

Answer:
double rotation

AVL tree insert

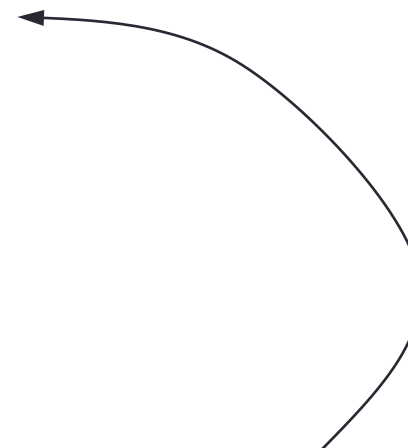
Let x be the **deepest** node where an imbalance occurs.
Four cases to consider.

The insertion is in the

- left subtree of the left child of x .
- right subtree of the left child of x .
- left subtree of the right child of x .
- right subtree of the right child of x .

Cases 1 & 4 are solved by a **single rotation**.

Cases 2 & 3 are solved by a **double rotation**.



This is the situation we
have here

Double rotation

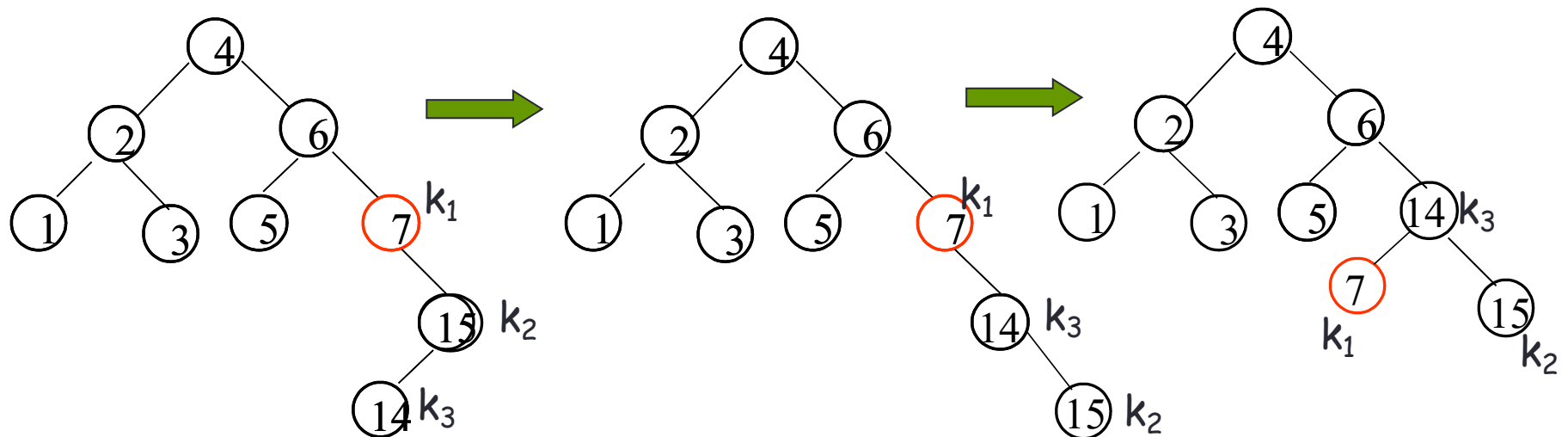
Let

- k_1 be the deepest node at which imbalance occurs
- k_2 and k_3 next two nodes along the path leading to the inserted node

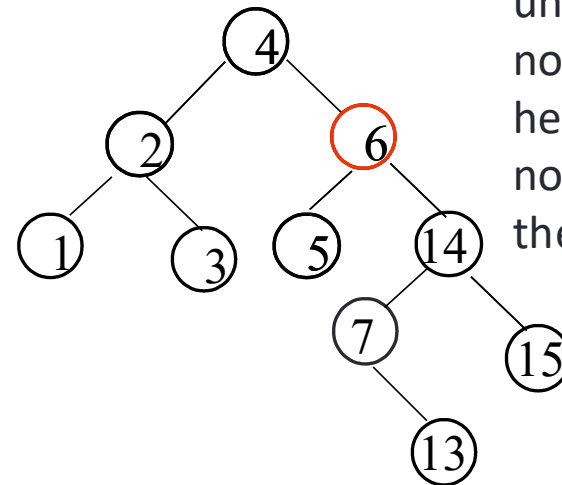
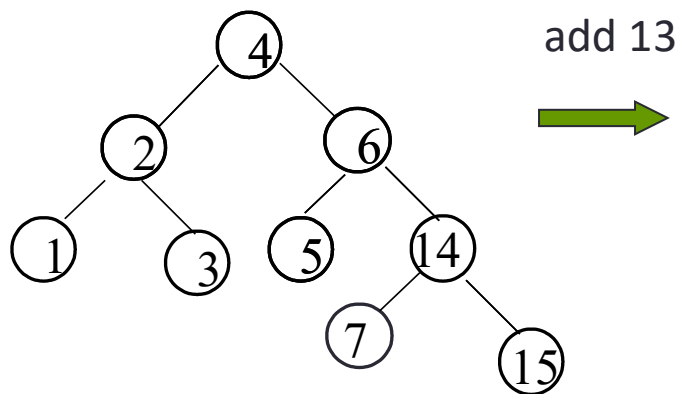
Then

- Rotate about k_2 and k_3
- Rotate about k_1 and k_3

Example: a right-left double rotation

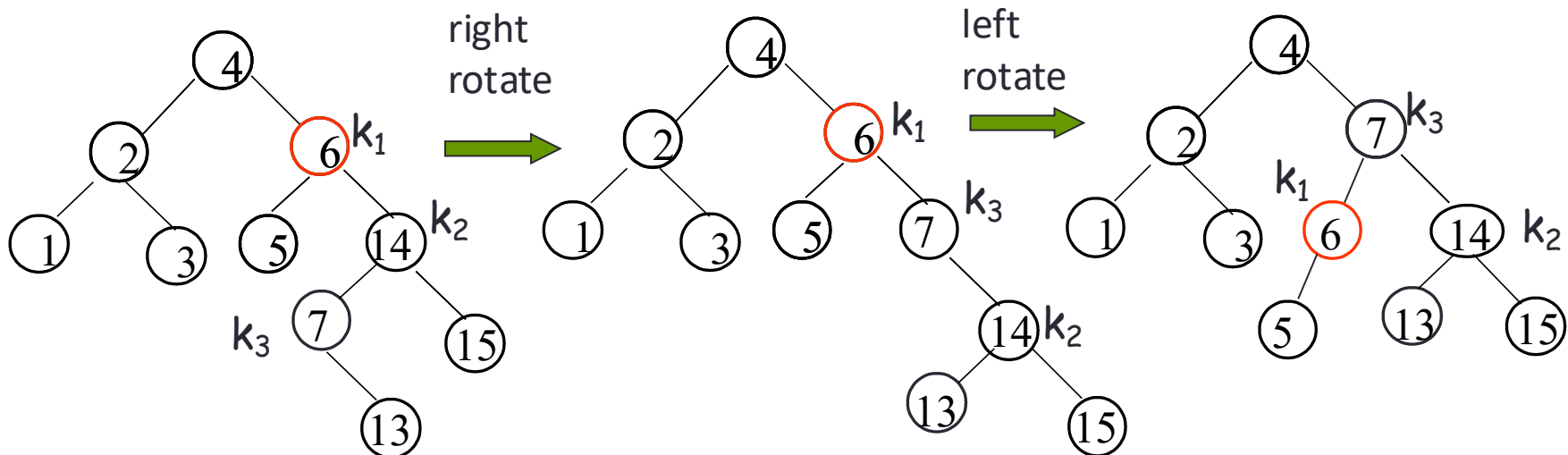


Continuing the example

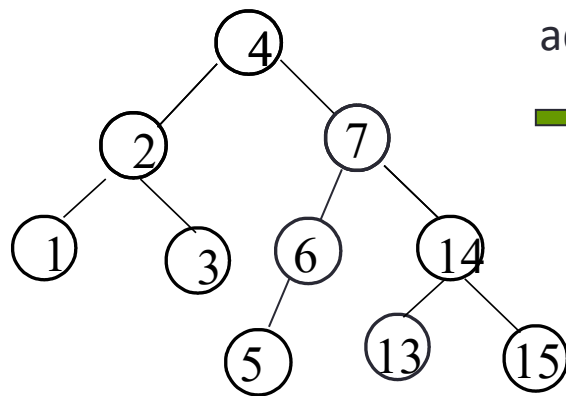


6 is unbalanced now (too heavy on the right). New node is in the left subtree of the right child so double rotation required.

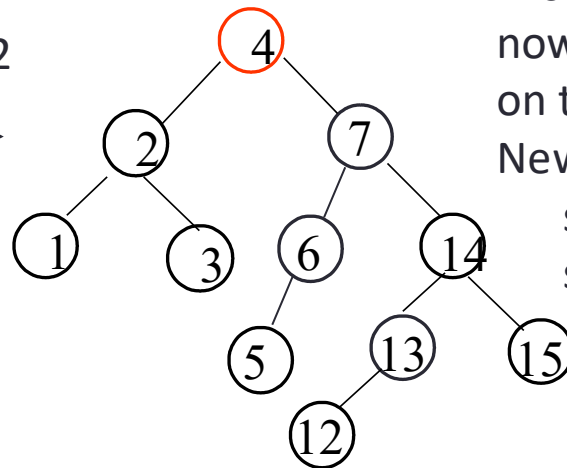
Again, right-left rotation required



Example continued



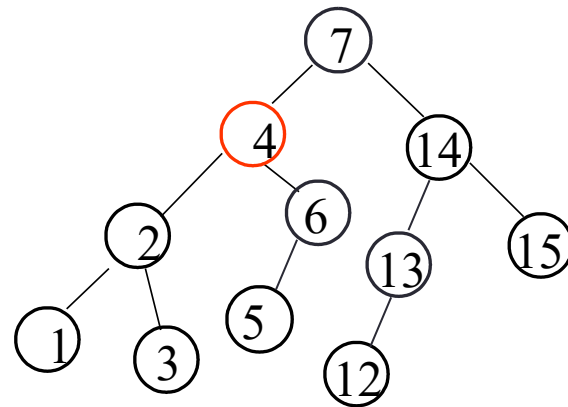
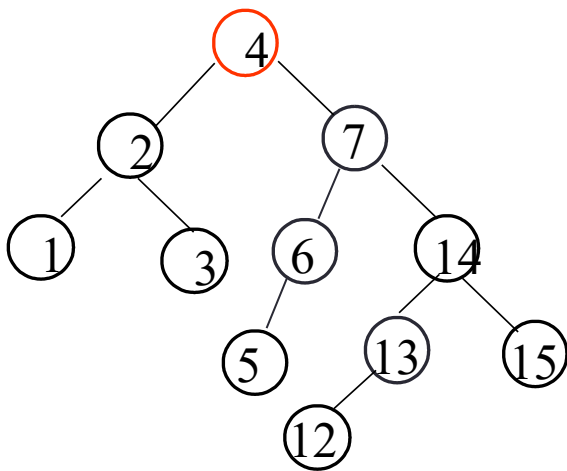
add 12



4 is unbalanced now (too heavy on the right).
New node is in the right subtree of the right child so single (left) rotation required.

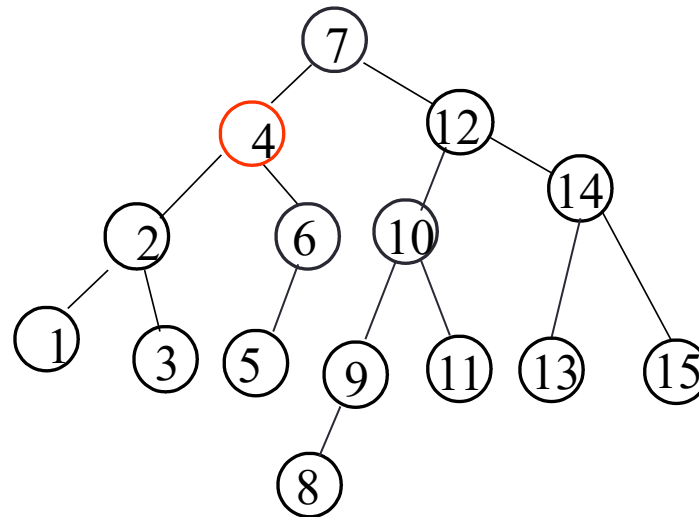
Left rotation.

7 **has** a left subtree to move



Example contd.

- Inserting 11, 10, 9 single rotation needed, inserting 8 requires no rotation. The resulting tree is:



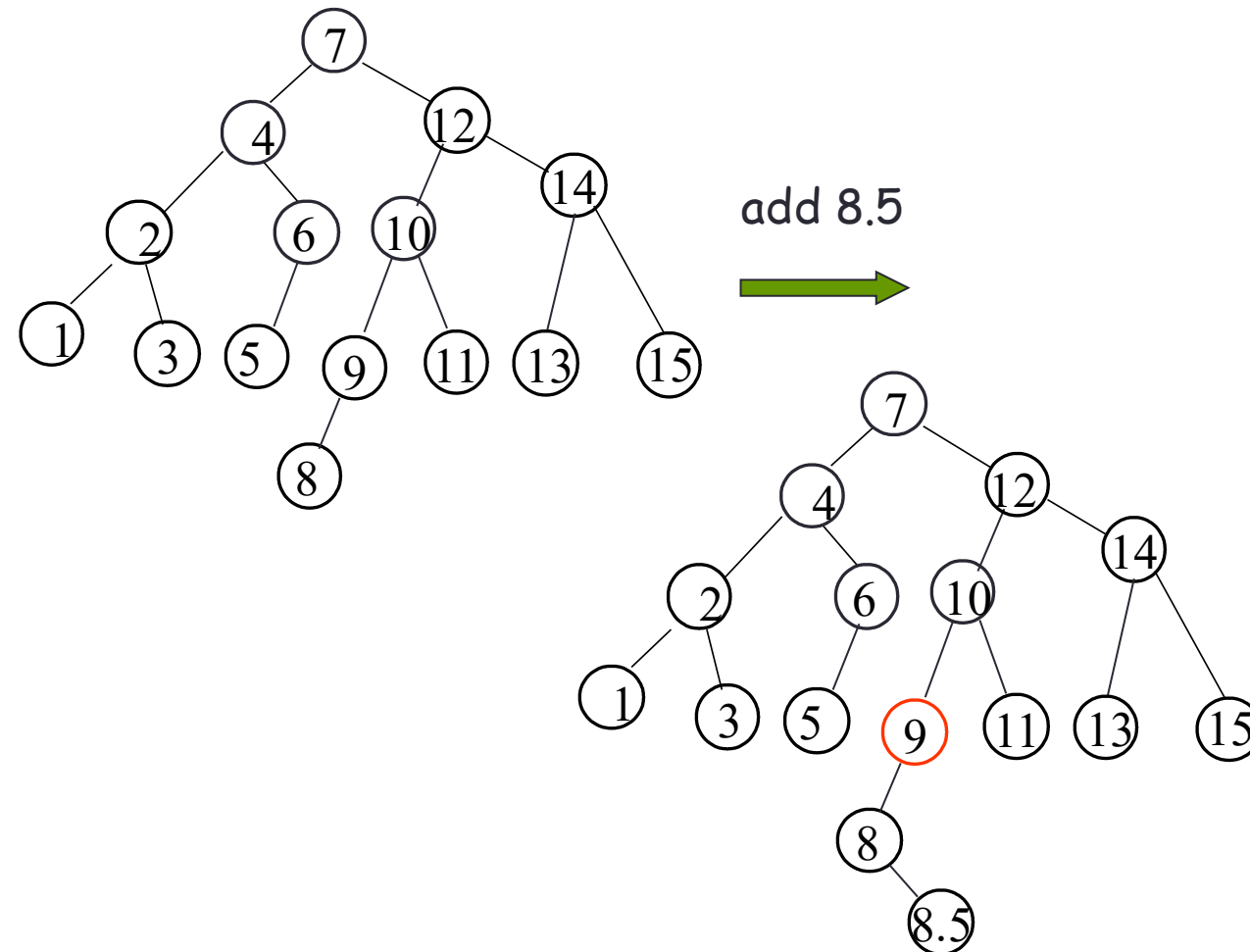
Example for you:

Show this is correct

But we haven't seen an example of a left-right double rotation!

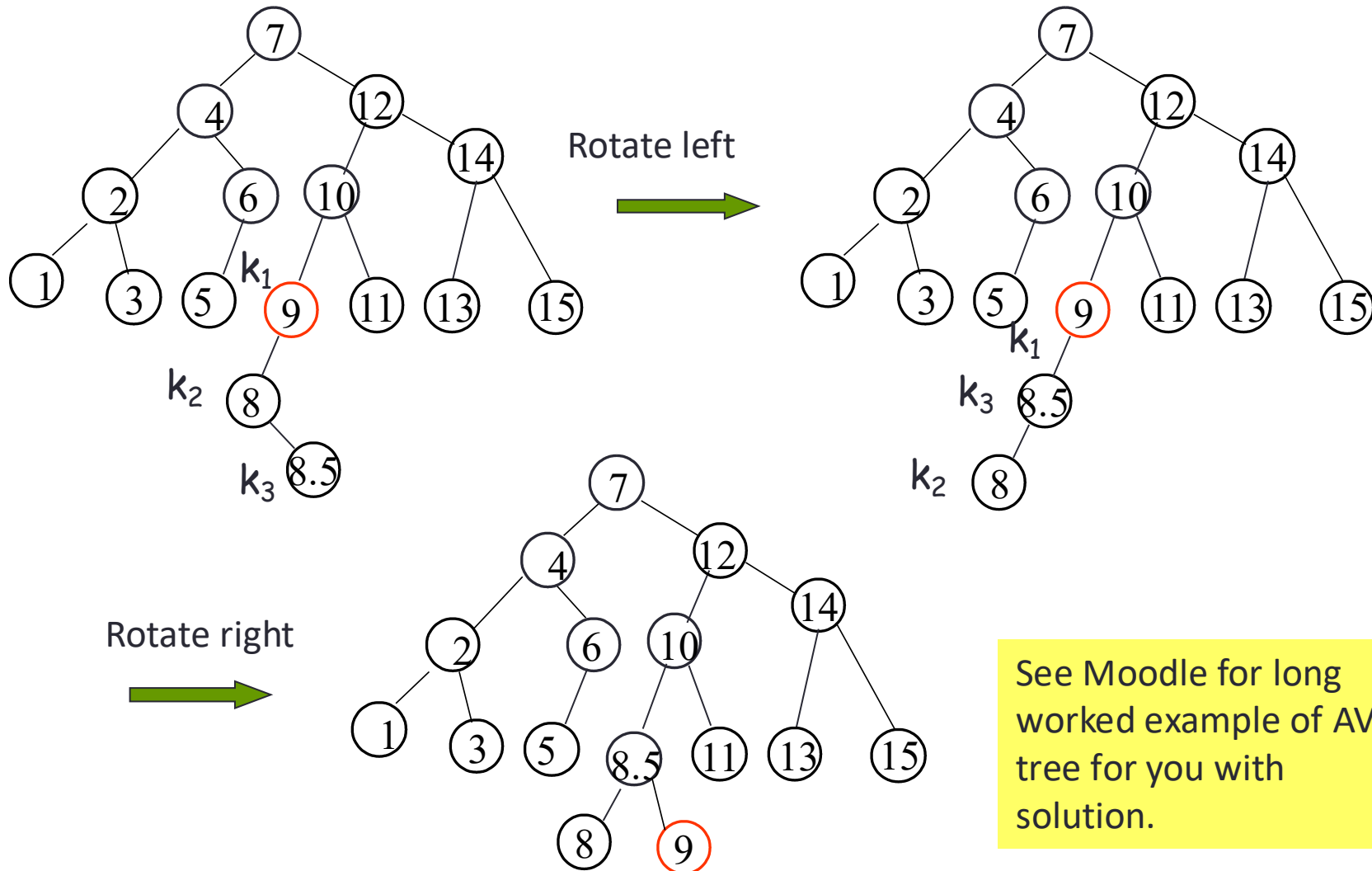
A left-right rotation

Suppose we want to add a node containing the value 8.5



9 is unbalanced now (too heavy on the left). New node is in the right subtree of the left child so double rotation required.

Left-right rotation contd.



See Moodle for long worked example of AVL tree for you with solution.

AVL implementation in Java

- Need to keep a record of the height of a node
 - So AVLNode defined thus:

```
public class AVLNode<E>{  
    //Instance variables:  
    private E element;  
    private AVLNode<E> left;  
    private AVLNode<E> right;  
    int height;  
    private AVLNode<E> parent;  
  
    //methods  
}
```

what's this?

Height of node p =
-1 if p is null
0 if p is leaf
 $\text{Max}(\text{height}(p.\text{left}), \text{height}(p.\text{right})) + 1$ otherwise.
Height of AVLTree = height of root.

It is necessary to also keep track of the parent of a node (to find the deepest unbalanced node), but we won't look at this in detail.

AVL implementation contd.

Need to include methods:

```
/**return height of tree rooted at p*/  
public int height(AVLNode<E> p){ }
```

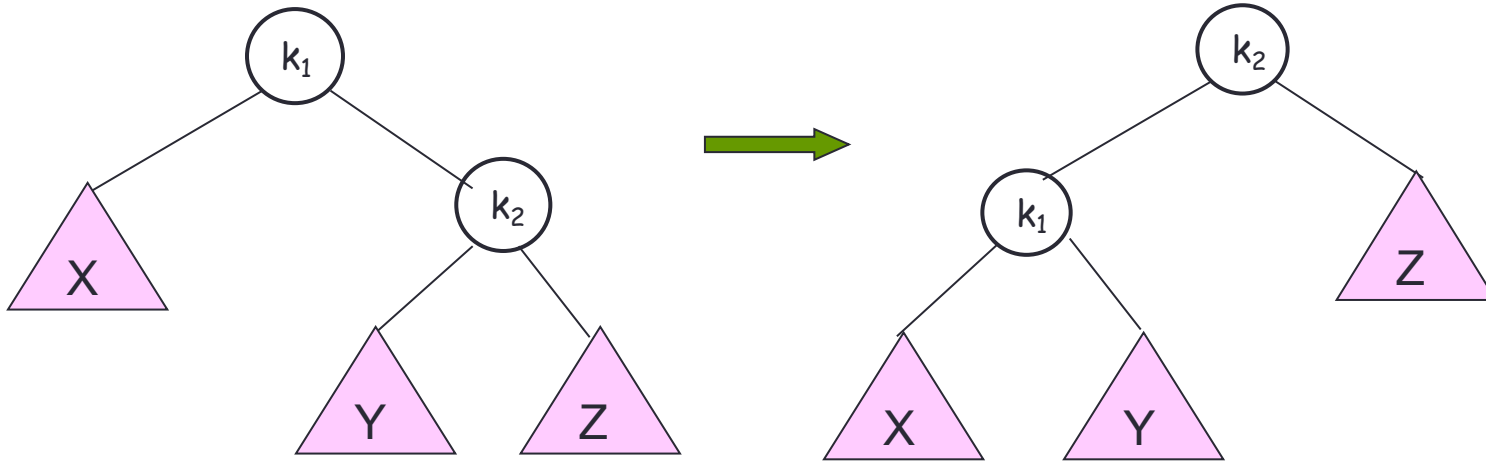
```
/**reset heights of all nodes starting at p */  
public void resetHeights(AVLNode<E> p){}
```

Java implementation of AVLTree cont.

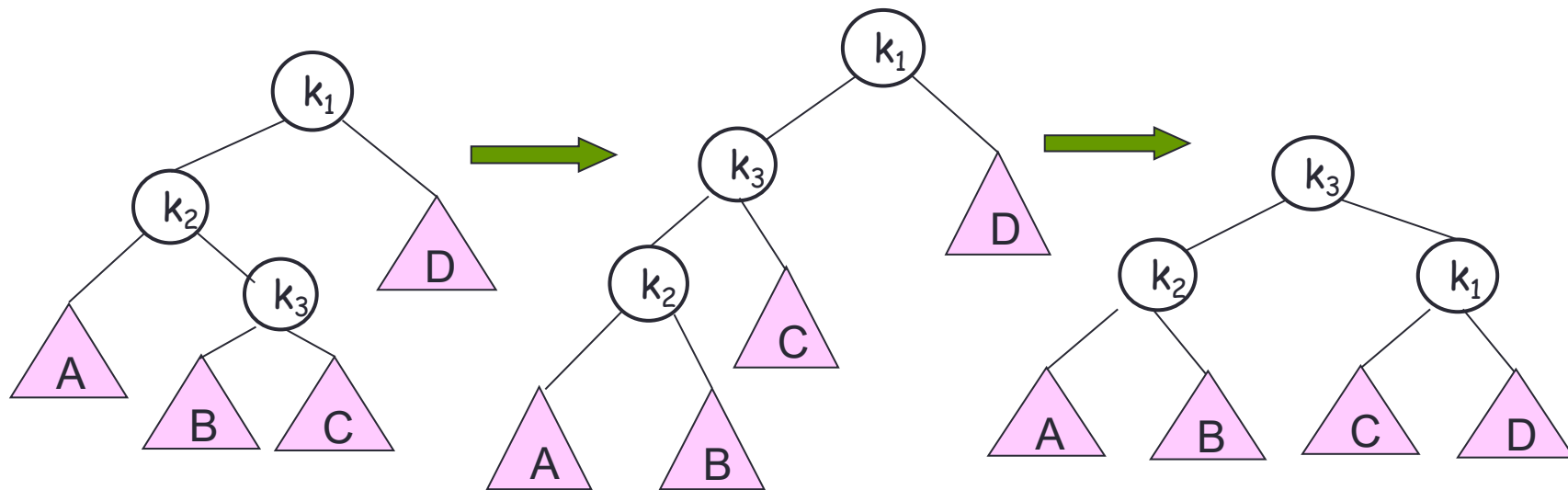
- Insert method needs to call other methods
 - sRotateLeft
 - sRotateRight
 - dRotateLeftRight
 - dRotateRightLeft

We don't look at all of these here, but will look at sRotateLeft and dRotateLeftRight

Delete method left as an exercise



```
public AVLNode<E> sRotateLeft(AVLNode<E> k1) {
    AVLNode<E> k2=k1.getRight();
    k1.setRight(k2.getLeft());
    k2.setLeft(k1);
    k1.setHeight(max(height(k1.getLeft()),
                      height(k1.getRight()))+1);
    k2.setHeight(max(height(k2.getRight()),
                      height(k1))+1);
    return k2;
}
```



```

public AVLNode<E> dRotateLeftRight (AVLNode<E> k1) {
    AVLNode<E> k2=k1.getLeft();
    AVLNode<E> k3=k2.getRight();
    k1.setLeft(sRotateLeft(k2));
    sRotateRight(k1);
    return k3;
}

```

When do I need to do a rotation?

During recursive implementation of insert for AVL tree (value e):

When insertion has been performed to a left subtree of node p :

- if height of subtrees of p differ by 2, need rotation:

 - if the new root of left subtree $< e$ need a single right rotation

 - else need a double left right rotation

Similar argument for when insertion has been performed to a right subtree of node p .

Only need to reset the heights of the tree when new node added, but no rotation required.

Partial Implementation available on Moodle